GPIO Peripheral

IP Specification, Bug Fixes and Verification

RISCFORGE 1TOPS | APB[3] | IP-GPIO-v3.0

Introduction

This document covers the GPIO peripheral of the RISCFORGE 1TOPS SoC: what the IP is, how software uses it, the RTL bugs found while verifying it, the changes made to fix them, and the verification that was run against the fixed design.

It supersedes the GPIO subsection of ARCH-INTEG for everything GPIO-specific. In particular the register offsets in Register Map are taken from the generated register package `gpio_reg_pkg.sv` and are the ones software must use.

Section	Contents
IP Overview	What the IP is, files, ports, SoC integration
Register Map	Register map and field encodings
Functional Description	Output path, input path, interrupts, cycle timing
Programming Guide	Programming guide with worked C sequences
Bugs Found	The six RTL bugs found
Changes Made	Every RTL change made, and what it changes for software
Verification	Testbench, test cases and regression history
Lint — JasperGold Superlint	JasperGold Superlint results and waivers
Verification Status	What is verified, what is not, and what remains open
Appendix: Superlint Script	The Superlint Tcl script

IP Overview

What the IP Is

The GPIO is the PULP-platform GPIO peripheral from ETH Zürich and the University of Bologna (author Manuel Eggimann, Solderpad Hardware License 0.51). Its register file is generated by the lowRISC `reggen` tool, and it reaches the APB bus through the PULP `register_interface` library.

- **32 pins**, each independently configurable as input only, push-pull output, or one of two open-drain output modes.
- **Atomic output updates** through write-only `GPIO_SET`, `GPIO_CLEAR` and `GPIO_TOGGLE` registers, so software never needs a read-modify-write to change one pin.
- **Synchronised inputs**: every pin passes through a 2-flop synchroniser, and sampling can be switched off per pin with `GPIO_EN`.

- **Four interrupt types per pin**: rising edge, falling edge, level high and level low, each individually enabled, each with its own sticky status register.
- **Two interrupt outputs**: one global line that goes to the core, and one line per pin. Each can be configured to follow live events or latched status.
- **Zero-wait-state APB slave**: PREADY is high in the first ACCESS cycle of every transfer.

Source Files

File	Content
`gpio_apb_wrap.sv`	APB wrapper; the module the SoC instantiates. Holds the sub-word write guard added in this revision.
`gpio.sv`	Core logic: output modes, SET/CLEAR/TOGGLE, interrupt generation and status. Also holds the `gpio_intf` wrapper.
`gpio_reg_top.sv`	`reggen`-generated register file.
`gpio_reg_pkg.sv`	`reggen`-generated package: `GPIOCount`, register offsets, `reg2hw`/`hw2reg` structs.
`gpio_input_stage_no_clk_gates.sv`	Per-pin input stage used in this SoC. Declares module `gpio_input_stage`.
`gpio_input_stage.sv`	Clock-gated variant of the same module. **Not compiled**: it needs `tc_clk_gating`, which no cell library provides in this flow.
`gpio_axi_lite_wrap.sv`	AXI-Lite wrapper. Not used in this SoC.

Dependencies outside the GPIO directory: `apb_to_reg.sv`, `reg_intf.sv` and `prim_subreg*.sv` from `SOC_FILES/include/register_interface/`, `apb_pkg.sv`, and `sync.sv` from `common_cells`.

Ports of `gpio_apb_wrap`

Port	Dir	Width	Function
`clk_i`	in	1	Peripheral clock
`rst_ni`	in	1	Asynchronous active-low reset
`gpio_in`	in	32	Pad inputs, asynchronous
`gpio_out`	out	32	Pad output values; always equals `GPIO_OUT`
`gpio_tx_en_o`	out	32	Pad output-buffer enable: 1 drives the pad, 0 releases it
`gpio_in_sync_o`	out	32	Synchronised inputs; same value as `GPIO_IN`
`global_interrupt_o`	out	1	Combined interrupt of all 32 pins
`pin_level_interrupts_o`	out	32	One interrupt line per pin
`apb_req_i`	in	struct	APB request; `pstrb` is used by the sub-word guard
`apb_rsp_o`	out	struct	APB response: `prdata`, `pready`, `pslverr`

Parameters: `ADDR_WIDTH` = 32, `DATA_WIDTH` = 32, `apb_req_t`, `apb_rsp_t`. The pin count is the local parameter `NrGPIOs` = `gpio_reg_pkg::GPIOCount` = 32; changing it requires regenerating the register file.

SoC Integration

Item	Value
APB slot	APB[3] of `axi_lite_to_apb`

Item	Value
Address window	0x4000_3000 – 0x4000_37FF (2 KB). Reduced from 4 KB in this revision; see B6.
Clock	clk_i
Reset	periph_rst_n from the bootloader, released with the core
Interrupt to core	global_interrupt_o → fast_irq_i[0] → mip[16], mcause 16
Per-pin interrupts	pin_level_interrupts_o brought out as a subsystem port; not connected to the core
Pads	gpio_in_i, gpio_out_o, gpio_tx_en_o brought out as subsystem ports

Register Map

Offsets are relative to 0x4000_3000. All registers are 32 bits and must be accessed with 32-bit, word-aligned stores (see Programming Guide).

Offset	Name	Access	Reset	Description
0x000	INFO	R	0x820	[9:0] pin count (32), [19:10] version (2)
0x004	CFG	R/W	0	[0] global interrupt mode, [1] per-pin interrupt mode
0x008	GPIO_MODE_0	R/W	0	2-bit mode for pins 0–15
0x00C	GPIO_MODE_1	R/W	0	2-bit mode for pins 16–31
0x080	GPIO_EN	R/W	0	Input sampling enable, one bit per pin
0x100	GPIO_IN	R	0	Synchronised input value
0x180	GPIO_OUT	R/W	0	Output value
0x200	GPIO_SET	W	—	Write 1 to set the GPIO_OUT bit; reads 0
0x280	GPIO_CLEAR	W	—	Write 1 to clear the GPIO_OUT bit; reads 0
0x300	GPIO_TOGGLE	W	—	Write 1 to invert the GPIO_OUT bit; reads 0
0x380	INTRPT_RISE_EN	R/W	0	Rising-edge interrupt enable
0x400	INTRPT_FALL_EN	R/W	0	Falling-edge interrupt enable
0x480	INTRPT_LVL_HIGH_EN	R/W	0	Level-high interrupt enable
0x500	INTRPT_LVL_LOW_EN	R/W	0	Level-low interrupt enable
0x580	INTRPT_STATUS	R/W1C	0	OR of the four status registers. Writing 1 to a bit clears all four status bits of that pin.
0x600	INTRPT_RISE_STATUS	R/W1C	0	Rising edge seen
0x680	INTRPT_FALL_STATUS	R/W1C	0	Falling edge seen
0x700	INTRPT_LVL_HIGH_STATUS	R/W1C	0	High level seen
0x780	INTRPT_LVL_LOW_STATUS	R/W1C	0	Low level seen

Important

The GPIO subsection of ARCH-INTEG v6.0 lists these registers at consecutive offsets (0x010, 0x014, ...). Those offsets are wrong. The generated register file places each register on a 0x80 boundary from GPIO_EN onward, as in the table above. A write to 0x4000_3010 is an unmapped access and returns PSLVERR. ARCH-INTEG also describes GPIO_EN as an output enable; it is an *input sampling* enable. Output drive is controlled by GPIO_MODE.

Any offset from 0x000 to 0x7FF not listed above is unmapped: a read or write returns PSLVERR, a read returns 0xFFFF_FFFF, and no register changes.

GPIO_MODE Encoding

Pin n uses bits [2n+1:2n] of GPIO_MODE_0 for $n < 16$, and bits [2(n-16)+1:2(n-16)] of GPIO_MODE_1 for $n \geq 16$.

Value	Mode	gpio_tx_en_o[n]	Use
00	Input only	0	Pad never driven
01	Push-pull	1	Pad always driven with GPIO_OUT[n]
10	Open drain 0	GPIO_OUT[n]	Drives only when the output is 1; released otherwise
11	Open drain 1	~GPIO_OUT[n]	Drives only when the output is 0; released otherwise (classic open drain, e.g. I2C-style wired-AND)

gpio_out[n] always equals GPIO_OUT[n] in every mode; the mode only controls whether the pad buffer is enabled.

CFG Encoding

Bit	Field	0	1
[0]	glbl_intrpt_mode	global_interrupt_o follows live events: an edge gives a 1-cycle pulse, a level holds while present	global_interrupt_o is the OR of all latched status bits; stays high until cleared
[1]	pin_lvl_intrpt_mode	Each pin_level_interrupts_o[n] follows live events	Each pin_level_interrupts_o[n] follows that pin's latched status

Note

For the core interrupt, set CFG[0] = 1. With CFG[0] = 0 an edge interrupt is a single-cycle pulse on fast_irq_i[0], and correct behaviour then depends on how the core latches mip[16] — that path has not been verified at SoC level.

Functional Description

Output Path

GPIO_OUT holds the output value of every pin. It can be written directly, or changed a bit at a time through GPIO_SET, GPIO_CLEAR and GPIO_TOGGLE: writing 1 to bit n of one of these acts on pin n only, and writing 0 has no effect. A write takes effect on the clock edge that completes the APB ACCESS phase — there is no extra latency.

Input Path

Each pin has its own gpio_input_stage: a 2-flop synchroniser followed by one sample register (serial_q). The synchroniser runs continuously; the sample register only updates while GPIO_EN[n] = 1. So:

- With GPIO_EN[n] = 1, GPIO_IN[n] follows the pin three clock edges later.

- With GPIO_EN[n] = 0, GPIO_IN[n] holds the last sampled value — 0 if the pin has never been enabled since reset.

Edges are detected by comparing the synchroniser output with the sample register: rising edge = serial & ~serial_q, falling edge = ~serial & serial_q. Both are qualified with GPIO_EN[n] (see B1).

Interrupts

Each pin has four interrupt sources. A source raises an event only when its enable bit is set and GPIO_EN[n] = 1:

Source	Event	Status behaviour
Rising edge	1 cycle, on a 0→1 sample	Sets INTRPT_RISE_STATUS[n]; held until cleared
Falling edge	1 cycle, on a 1→0 sample	Sets INTRPT_FALL_STATUS[n]; held until cleared
Level high	Every cycle the sample is 1	Sets INTRPT_LVL_HIGH_STATUS[n]; re-sets on the next cycle if cleared while the level is still present
Level low	Every cycle the sample is 0	Sets INTRPT_LVL_LOW_STATUS[n]; same re-set behaviour

INTRPT_STATUS[n] is the OR of the four status bits of pin n. Clearing is write-1-to-clear: a 1 in bit n of a per-type status register clears that one bit; a 1 in bit n of INTRPT_STATUS clears all four status bits of pin n. An event arriving on the same clock edge as a clear is kept, not lost (see B4, B5).

Cycle Timing

“Edge” means rising clock edge after the pin changes. Every row is measured by TC10, TC13 or TC14 except the level status latency, which is taken from the RTL.

Event	Latency	Note
Pin change → GPIO_IN / gpio_in_sync_o	3 edges	Pin enabled
Pin edge → edge interrupt output (CFG bit = 0)	2 edges	1-cycle pulse
Pin edge → edge status bit set	3 edges	
Pin level → level interrupt output (CFG bit = 0)	3 edges	Held while the level is present
Pin level → level status bit set	4 edges	From RTL; not measured
APB write → gpio_out / gpio_tx_en_o	0	Updates on the ACCESS commit edge
APB wait states	0	PREADY high in the first ACCESS cycle

A pulse shorter than one clock period that does not straddle a clock edge is never sampled and raises no interrupt (TC13).

Programming Guide

Rules

Important

1. **Use 32-bit stores only.** A byte or half-word store to any GPIO register is rejected with PSLVERR and changes nothing. The AXI bridge returns this as SLVERR; per ARCH-INTEG the core then takes a store access fault. Sub-word *loads* are fine.
2. **Set `GPIO_EN` before reading a pin.** A disabled pin reads its last sampled value, not the pad.
3. **Clear `INTRPT_STATUS` after enabling pins, before enabling their interrupts.** If a pin's level changed while it was disabled, the first clock after enabling it reports one edge.
4. **Do not use `0x4000_3800` – `0x4000_3FFF`.** This range is outside the GPIO window and returns SLVERR.
5. **Clear a level interrupt only after removing its cause.** Clearing it while the level is still present has no lasting effect.

Register Definitions

Listing 1: Register access macros

```
#define GPIO_BASE                 0x40003000u
#define GPIO_REG(off)             (*(volatile uint32_t *)(GPIO_BASE + (off)))

#define GPIO_INFO                 GPIO_REG(0x000)
#define GPIO_CFG                  GPIO_REG(0x004)
#define GPIO_MODE_0               GPIO_REG(0x008)   /* pins  0..15 */
#define GPIO_MODE_1               GPIO_REG(0x00C)   /* pins 16..31 */
#define GPIO_EN                   GPIO_REG(0x080)
#define GPIO_IN                   GPIO_REG(0x100)
#define GPIO_OUT                  GPIO_REG(0x180)
#define GPIO_SET                  GPIO_REG(0x200)
#define GPIO_CLEAR                GPIO_REG(0x280)
#define GPIO_TOGGLE               GPIO_REG(0x300)
#define GPIO_INTRPT_RISE_EN       GPIO_REG(0x380)
#define GPIO_INTRPT_FALL_EN       GPIO_REG(0x400)
#define GPIO_INTRPT_LVL_HIGH_EN   GPIO_REG(0x480)
#define GPIO_INTRPT_LVL_LOW_EN    GPIO_REG(0x500)
#define GPIO_INTRPT_STATUS        GPIO_REG(0x580)
#define GPIO_INTRPT_RISE_STATUS   GPIO_REG(0x600)
#define GPIO_INTRPT_FALL_STATUS   GPIO_REG(0x680)
#define GPIO_INTRPT_LVL_HIGH_STATUS GPIO_REG(0x700)
#define GPIO_INTRPT_LVL_LOW_STATUS GPIO_REG(0x780)

#define GPIO_MODE_INPUT           0x0u
#define GPIO_MODE_PUSH_PULL       0x1u
#define GPIO_MODE_OPEN_DRAIN0     0x2u
#define GPIO_MODE_OPEN_DRAIN1     0x3u
```

Driving Outputs

Listing 2: Pins 0–7 as push-pull outputs

```
GPIO_OUT    = 0x00000000;         /* choose the value before enabling the drivers   */
GPIO_MODE_0 = 0x00005555;         /* pins 0..7 = 01 (push-pull), pins 8..15 = input */

GPIO_SET    = 1u << 3;            /* pin 3 high, other pins untouched               */
GPIO_CLEAR  = 1u << 3;            /* pin 3 low                                      */
GPIO_TOGGLE = 0x000000F0;         /* invert pins 4..7 in one write                  */
```

Reading Inputs

Listing 3: Pins 8–15 as inputs

```
GPIO_EN |= 0x0000FF00;            /* enable sampling on pins 8..15                  */
/* GPIO_IN is valid three clock cycles after the pad settles */
uint32_t pins = (GPIO_IN >> 8) & 0xFF;
```

Edge Interrupt

Listing 4: Rising-edge interrupt on pin 12, sticky mode

```
GPIO_EN             |= 1u << 12;   /* 1. sample the pin                                */
GPIO_INTRPT_STATUS   = 1u << 12;   /* 2. drop any edge reported by enabling it         */
GPIO_CFG             = 0x1;        /* 3. global line = OR of latched status            */
GPIO_INTRPT_RISE_EN |= 1u << 12;   /* 4. arm the pin                                   */
/* 5. core side: set mie bit 16 and mstatus.MIE                                       */

void gpio_isr(void)                /* mcause = 16                                      */
{
    uint32_t pending = GPIO_INTRPT_STATUS;       /* which pins                         */
    uint32_t rise    = GPIO_INTRPT_RISE_STATUS;  /* which kind, if more than one armed */
    /* ... handle pending / rise ... */
    GPIO_INTRPT_STATUS = pending;                /* W1C: acknowledge exactly what was read */
}
```

Acknowledging exactly the bits that were read, rather than writing `0xFFFF_FFFF`, means an event that arrives on another pin while the handler runs stays pending and re-raises the interrupt when the handler returns.

Bugs Found

Six RTL bugs were found. B1–B5 are fixed in the GPIO and verified by the block testbench. B6 is fixed in the SoC address map and has not yet been verified at SoC level.

ID	Bug	Severity	Status
B1	Disabled pin raises edge interrupts; `global_interrupt_o` stuck high	High	Fixed, verified (TC19)
B2	Disabled pin raises a level-low interrupt from its frozen 0	Medium	Fixed, verified (TC19)
B3	Byte and half-word stores zero the rest of the register	High	Fixed, verified (TC22)
B4	Edge lost when a per-type status clear lands on the same edge	Low	Fixed, verified (TC18)
B5	Edge lost when an `INTRPT_STATUS` clear lands on the same edge	Low	Fixed, verified (TC18)
B6	Offsets `0x800 - 0xFFF` alias onto the real registers	Medium	Fixed in SoC map; SoC-level check pending

B1 — Disabled Pin Raises Edge Interrupts

The Issue. In `gpio_input_stage_no_clk_gates.sv` the 2-flop synchroniser keeps running while `en_i` = 0, but `serial_q` is frozen. The edge outputs were `r_edge_o = serial & ~serial_q` and `f_edge_o = ~serial & serial_q`, with no enable term. So whenever a disabled pin's level differs from its frozen sample, the edge output stays at 1 for as long as the difference lasts. The clock-gated variant `gpio_input_stage.sv` does not have this problem because it stops the synchroniser too; the defect was introduced when the no-clock-gate variant was substituted for it.

Impact. A pin with `GPIO_EN[n]` = 0 and rise or fall interrupts enabled latches status, and with `CFG[0]` = 0 holds `global_interrupt_o` high continuously. That line is `mip[16]`: the core re-enters the GPIO handler on every return, and the system stops making progress. TC19 measured the line high for 6 consecutive cycles, where the specification is a 1-cycle pulse.

Fix. Qualify both edge outputs with `en_i`.

B2 — Disabled Pin Raises a Level-Low Interrupt

The Issue. A disabled pin's `GPIO_IN` bit is frozen at its reset value 0. The level-low term in `gpio.sv` is `~s_gpio_in_sync & s_gpio_low_intrpt_mask`, with no enable term, so a level-low interrupt fires even when the pad is high.

Impact. A spurious, permanent level-low interrupt on any disabled pin that has level-low enabled.

Fix. Mask both level terms with the per-pin `GPIO_EN` bit.

B3 — Sub-Word Stores Wipe the Register

The Issue. `apb_to_reg.sv` has no strobe port and drives `reg_o.wstrb = '1`, so `pstrb` never reaches the register file. `gpio_reg_top` writes all 32 bits of `reg_wdata` regardless of `reg_be`; it uses `reg_be` only to raise `wr_err` against `GPIO_PERMIT`, and with `reg_be` always `4'hF` that check can never fire. The core's `ls_unit` places a byte store in its lane and leaves the other three bytes 0.

Impact. `*(volatile uint8_t *)0x40003180 = 0xFF` drives pins 0–7 high and silently drives pins 8–31 low. On a W1C status register a byte store also clears pending interrupt bits it did not intend to touch.

Fix. Reject sub-word writes in `gpio_apb_wrap`. A write whose `pstrb` is not all ones is withheld from the bridge and answered with PSLVERR. Passing the strobes through to the register file would have had the same effect, but needed either a modified vendored bridge or a forked copy; the wrapper change touches only the GPIO. A read-modify-write in the bridge was rejected: on the W1C status registers it would read the pending bits and write them back, clearing every pending interrupt.

B4 — Edge Lost on a Coincident Per-Type Clear

The Issue. `prim_subreg_arb.sv` resolves a W1C field as `wr_data = (de ? d : q) & (we ? ~wd : '1)`. When a hardware set (`de` = 1) and a software clear (`wd` = 1) land on the same clock edge, the `~wd` term forces 0 and the event is never recorded.

Impact. One clock window per clear. With `CFG[0]` = 1 the edge vanishes completely; with `CFG[0]` = 0 the core gets the interrupt pulse but finds no status bit set. The window opens on every acknowledge, so it matters for fast or bursty inputs — pulse counters, encoders, handshake lines — not for buttons.

Fix. Hardware set wins: `wr_data = de ? d : (q & (we ? ~wd : '1))`.

B5 — Edge Lost on a Coincident `INTRPT_STATUS` Clear

The Issue. A write to `INTRPT_STATUS` never reaches the W1C arbiter as a software write. `gpio.sv` turns it into a hardware write of `d = 0`, `de = 1` on all four status registers of the pin, and that branch ignores any event arriving in the same cycle. B4's fix therefore does not cover it.

Impact. Same as B4, on the aggregated clear path — which is the one the recommended interrupt handler uses.

Fix. In the clear branch, write each status bit with the incoming event instead of 0.

B6 — Address Aliasing

The Issue. `gpio_reg_top` decodes only 11 address bits (`BlockAw` = 11), so `reg_addr` is `paddr[10:0]`. The SoC decoded a 4 KB window to the GPIO, so offsets `0x800 - 0xFFF` folded back onto `0x000 - 0x7FF`.

Impact. TC05 wrote `0xDEADBEEF` to offset `0x980` and it appeared on the pins through `GPIO_OUT` (`0x180`). A stray pointer into the upper half of the window silently changes pin state instead of faulting.

Fix. Narrow `apb_addr_map[3]` to 2 KB. Addresses in the upper half then match no APB rule, `axi_lite_to_apb` never selects the GPIO, and it answers with SLVERR itself. The IP itself still aliases, so TC05 — which drives the GPIO directly with no SoC decoder — still reports it by design.

Issues Found in the Testbench Itself

Two further failures during verification were not RTL bugs in the original design:

Symptom	Cause and resolution
TC04: 22 unmapped accesses showed `pslverr = 0`	The BFM sampled PSLVERR in the same time step it drove PENABLE, before the combinational response had propagated, so it read the SETUP-phase value. The RTL was correct: the unmapped writes were blocked, and reads returned the unmapped default. Fixed by sampling one delta later.
11 SVA failures, `PSLVERR outside ACCESS phase`	The first version of the B3 guard was not qualified with PENABLE, so it drove PSLVERR during SETUP too. Legal APB, since PSLVERR is only sampled in ACCESS, but untidy and flagged by the assertion. Fixed by adding PENABLE to the guard.

Changes Made

#	File	Change	Fixes
1	`gpio_input_stage_no_clk_gates`	Edge outputs qualified with `en_i`	B1
2	`gpio.sv`	Level-high and level-low terms masked with `s_gpio_en`	B2
3	`gpio.sv`	`INTRPT_STATUS` clear branch keeps a coincident event	B5
4	`prim_subreg_arb.sv` (vendored)	W1C: hardware set wins over a coincident software clear	B4
5	`gpio_apb_wrap.sv`	Sub-word write guard, ACCESS phase only	B3
6	`riscforge_soc_subsystem.sv`	GPIO APB window reduced to 2 KB	B6
7	`apb_i2c.sv`	Comment only, no logic change: marks the same race in the I2C	—

1. Edge Qualification — `gpio_input_stage_no_clk_gates.sv`

```
// was: assign f_edge_o = (~serial) & serial_q;
//      assign r_edge_o = serial & (~serial_q);
assign f_edge_o = en_i & (~serial) & serial_q;
assign r_edge_o = en_i & serial & (~serial_q);
```

2. Level Masking — `gpio.sv`

```
logic [NrGPIOs-1:0] s_gpio_en;

assign s_gpio_high_intrpt =  s_gpio_in_sync & s_gpio_high_intrpt_mask & s_gpio_en;
assign s_gpio_low_intrpt  = ~s_gpio_in_sync & s_gpio_low_intrpt_mask  & s_gpio_en;

// inside gen_gpios:
assign s_gpio_en[gpio_idx] = s_reg2hw.gpio_en[gpio_idx].q;
```

3. Aggregated Clear Keeps the Event — gpio.sv

```
if (s_reg2hw.intrpt_status[gpio_idx].qe & (s_reg2hw.intrpt_status[gpio_idx].q == 1)) begin
  s_hw2reg.intrpt_rise_status[gpio_idx].d      = s_gpio_rise_intrpt[gpio_idx];  // was '0
  s_hw2reg.intrpt_rise_status[gpio_idx].de     = 1'b1;
  s_hw2reg.intrpt_fall_status[gpio_idx].d      = s_gpio_fall_intrpt[gpio_idx];  // was '0
  s_hw2reg.intrpt_fall_status[gpio_idx].de     = 1'b1;
  s_hw2reg.intrpt_lvl_high_status[gpio_idx].d  = s_gpio_high_intrpt[gpio_idx]; // was '0
  s_hw2reg.intrpt_lvl_high_status[gpio_idx].de = 1'b1;
  s_hw2reg.intrpt_lvl_low_status[gpio_idx].d   = s_gpio_low_intrpt[gpio_idx];  // was '0
  s_hw2reg.intrpt_lvl_low_status[gpio_idx].de  = 1'b1;
end
```

4. W1C Arbitration — prim_subreg_arb.sv

```
end else if (SWACCESS == "W1C") begin : gen_w1c
  assign wr_en   = we | de;
  // was: assign wr_data = (de ? d : q) & (we ? ~wd : '1);   // SW wins
  assign wr_data = de ? d : (q & (we ? ~wd : '1));           // HW set wins
```

Important

This is a change to a shared, vendored file: SOC_FILES/include/register_interface/prim_subreg_arb.sv.

- In this SoC it is instantiated only by gpio_reg_top; the GPIO is the only reggen IP in the build. No other IP is affected today.
- **It is only correct for single-bit W1C fields**, where de = 1 means an event on that exact bit. Every W1C field in gpio_reg_top is 1 bit wide. For a multi-bit W1C field it would resurrect bits the write was clearing.
- Re-check this branch before adding another reggen IP, and re-apply it after any refresh of the vendored PULP tree, which would silently revert it.

The change is commented in the file with the upstream line, the reason, and the single-bit restriction.

5. Sub-Word Write Guard — gpio_apb_wrap.sv

```
assign s_sub_word_wr = apb_req_i.psel & apb_req_i.penable & apb_req_i.pwrite &
                       ~(&apb_req_i.pstrb);

assign apb_rsp_o.prdata  = s_bridge_prdata;
assign apb_rsp_o.pready  = s_sub_word_wr ? 1'b1 : s_bridge_pready;
assign apb_rsp_o.pslverr = s_sub_word_wr ? 1'b1 : s_bridge_pslverr;

apb_to_reg i_abp_to_reg (
  .penable_i ( apb_req_i.penable & ~s_sub_word_wr ),
  .psel_i    ( apb_req_i.psel    & ~s_sub_word_wr ),
  .prdata_o  ( s_bridge_prdata   ),
  .pready_o  ( s_bridge_pready   ),
  .pslverr_o ( s_bridge_pslverr  ),
  // other ports unchanged
);
```

Reads are unaffected: the guard is qualified with pwrite. apb_to_reg.sv itself is unchanged, and no filelist changed.

6. GPIO Window — riscforge_soc_subsystem.sv

```
// was: end_addr: 32'h4000_4000
assign apb_addr_map[3] = '{idx: 32'd3, start_addr: 32'h4000_3000, end_addr: 32'h4000_3800}; // GPIO
```

What Changes for Software

Situation	Before	After
Byte or half-word store to a GPIO register	Other 24 or 16 bits zeroed silently	PSLVERR; register unchanged; store access fault
Access to `0x4000_3800 - 0x4000_3FFF`	Aliased onto real registers	SLVERR from the APB bridge
Disabled pin changes level	Could latch status and hold the interrupt line high	No interrupt
Pin enabled after its level changed while disabled	Edge already being reported continuously	One edge on the first cycle after enabling
Edge arrives on the same edge as a status clear	Event lost	Event kept
`INTRPT_STATUS` cleared while a level is still present	Status bit 0 for one cycle, then re-set	Status bit stays 1; no one-cycle dip on the interrupt line with `CFG[0]` $= 1$

Verification

Environment

Item	Value
Testbench	`ips/gpio-master/tb_gpio_apb_directed.sv`
Filelist	`ips/gpio-master/tb_gpio_apb_directed.f`, paths relative to `rtl/gpio-master` on the CentOS tree
Simulator	Cadence Xcelium 24.09-s001
DUT	`gpio_apb_wrap`, with the same APB struct types the SoC uses
Seed	1 (`+SEED=<n>` to change)
Run time	108,866 ns simulated, $\approx$ 5 s wall clock

Listing 5: Running the testbench

```
cd /home/iiita/MALAY_SOC/rtl/gpio-master
xrun -f tb_gpio_apb_directed.f
```

Methodology

- **Directed and constrained-random, self-checking, non-UVM.** Each test computes its own expected values inline; there is no reference model or scoreboard.
- **APB driver** drives on posedge + 1 ns, samples PRDATA and PSLVERR inside the ACCESS phase after the response settles, supports back-to-back transfers with PSEL held, and takes a byte-strobe argument.
- **Cycle-exact alignment.** Every task starts and ends at posedge + 1 ns, so the latency checks count edges exactly.
- **Sticky monitors** latch single-cycle interrupt pulses and measure how long `global_interrupt_o` stays high without a break.
- **Three result classes. PASS**, FAIL, and **BUG?** for behaviour that is legal for the RTL but likely wrong for the SoC. **BUG?** is counted separately so it cannot hide a real failure.
- **Watchdog** of 50 ms simulated time.

Assertions

Assertion	Property
a_slverr_in_access	PSLVERR is only high while PSEL and PENABLE are both high
a_pready_in_access	PREADY is high in every ACCESS cycle (zero wait states)
a_prdata_known	PRDATA is not X during a read ACCESS
a_outputs_known	No DUT output is X after reset

Test Cases and Final Result

Test	What It Checks	Result
TC01	Reset value of every output and all 19 registers; INFO = 0x820	PASS
TC02	9 RW registers: 76 patterns each (walking-1, walking-0, random); reserved CFG bits read 0; no crosstalk	PASS
TC03	INFO and GPIO_IN ignore writes; SET/CLEAR/TOGGLE read 0 with no side effect	PASS
TC04	11 unmapped offsets: PSLVERR on read and write; no register corrupted	PASS
TC05	Offsets 0x800 and 0x980 alias onto real registers (B6)	BUG? ×2
TC06	3 writes and 3 reads back-to-back with PSEL held high	PASS
TC07	All four pin modes: gpio_out and gpio_tx_en_o for fixed and random values, uniform and mixed modes	PASS
TC08	SET, CLEAR and TOGGLE walked over all 32 pins; zero writes; 20 multi-bit masks; open-drain tx_en; zero-latency commit	PASS
TC09	200 random MODE/OUT/SET/CLEAR/TOGGLE operations	PASS
TC10	Input latency exactly 3 edges, not 1 or 2	PASS
TC11	GPIO_EN: disabled pins hold their value; enable walked over all 32 pins	PASS
TC12	128 input vectors, a new one every clock cycle, sampled in order	PASS
TC13	Rise, fall and both-edge interrupts on pins 0, 31 and a random pin: 2-edge latency, 1-cycle pulse, sticky status, CFG = 3; all 32 pins; sub-cycle glitch rejected	PASS
TC14	Level-high and level-low on pins 0, 31 and a random pin: 3-edge latency, held while present, re-latch after clear; all 32 pins	PASS
TC15	W1C: only written bits clear; writing 0 clears nothing; aggregated clear clears all four types	PASS
TC16	CFG[0] and CFG[1] independent: sticky versus pulse on each output	PASS
TC17	With all enables 0, no interrupt under heavy pin activity; with one pin enabled, only that pin reports	PASS
TC18	Edge coincident with a per-type clear and with an INTRPT_STATUS clear survives; control case one cycle early (B4, B5)	PASS
TC19	Disabled pin: no edge status, interrupt stays a pulse, no level-low interrupt (B1, B2)	PASS
TC20	Asynchronous reset in the middle of an APB ACCESS phase; all registers return to reset values	PASS
TC21	300 random mixed output and input operations	PASS
TC22	Sub-word writes: 8 strobe patterns on GPIO_OUT, byte write to GPIO_SET and to a W1C status register, recovery back-to-back (B3)	PASS

Final result

312 PASSED / 0 FAILED / 2 BUG?

Both **BUG?** items are TC05, block-level aliasing, which the SoC address map handles and a block-level testbench cannot see.

Regression History

Run	State	Pass	Fail	BUG?	Open
1	Original RTL	294	2	7	TC04, TC05, TC18, TC19
2	Fixes 2–4 compiled; fix 1 and BFM fix not yet copied to CentOS	297	2	4	TC04, TC05, TC19 edge
3	All fixes 1–4 and BFM fix	301	0	2	TC05
4	TC22 added; guard without PENABLE	312	11	2	SVA, TC05
5	Guard qualified with PENABLE	312	0	2	TC05

Run 2 is a useful cross-check: exactly the fixes that were compiled moved their tests, and the two that were not compiled left theirs unchanged. Across all five runs no previously passing check regressed.

Lint — JasperGold Superlint

Setup

Item	Value
Tool	Cadence JasperGold Superlint, version 2025.06 FCS, 64-bit
Script	`lint.tcl`, reproduced in Appendix: Superlint Script
Filelist	`gpio_lint.f`: the RTL of the Xcelium run without the testbench, plus `gpio_lint_top.sv`
Top	`gpio_lint_top`
Clock / reset	`clk_i` / `~rst_ni`
Passes	1: LINT domain only (sign-off, same rule set as the I2C flow). 2: all rule domains (diagnostic)
Reports	`reports/gpio_superlint_lint.txt`, `reports/gpio_superlint_all_domains.txt`

Listing 6: Running the lint

```
cd /home/iiita/MALAY_SOC/rtl/gpio-master
jg -superlint lint.tcl
```

Why `gpio_lint_top.sv` Was Created

The problem. Superlint needs one top module it can elaborate. The natural choice is `gpio_apb_wrap`, the module the SoC instantiates, but it cannot be elaborated on its own. Its APB ports are declared with type parameters:

```
parameter type apb_req_t = logic,   // default: a single bit
parameter type apb_rsp_t = logic,
...
input  apb_req_t apb_req_i,
output apb_rsp_t apb_rsp_o
```

The real struct types are supplied only by whoever instantiates the wrapper. Elaborated as the top, `apb_req_i` is a single `logic` bit, and every field access inside the wrapper — `apb_req_i.penable`, `apb_req_i.pstrb`, `apb_rsp_o.prdata` — refers to a field that does not exist, so elaboration fails.

Alternatives that were rejected.

- **Lint the whole SoC** with `riscforge_soc_subsystem` as top. The GPIO would be elaborated correctly, but the report would be dominated by every other IP, the core and the interconnect, and would no longer be a GPIO sign-off.
- **Use** `gpio_apb_wrap_intf`, the interface-based variant in the same file. It needs the PULP `APB` SystemVerilog interface, which is not present in this source tree, and it is not the variant the SoC uses.

What the file does. `gpio_lint_top` is a thin wrapper with no logic of its own:

- It declares the APB request and response structs with the same `APB_TYPEDEF_REQ_T` / `APB_TYPEDEF_RESP_T` macros and the same widths the SoC uses: 32-bit address and data, 4-bit strobe.
- It instantiates `gpio_apb_wrap` with the same parameters as APB[3] in `riscforge_soc_subsystem`, so the netlist being linted is the SoC's GPIO configuration, not a variant of it.
- It exposes every APB field and pad signal as a flat top-level port, and packs and unpacks the structs with continuous assignments.
- It ties `PPROT` to `3'b000` rather than exposing it: the GPIO does not use it, and an exposed unused input would only add a spurious warning.

Scope. The file is for lint only. It lives in the `gpio-master` run directory, is listed last in `gpio_lint.f`, and appears in no SoC or simulation filelist.

Effect on the results. Because the wrapper's ports are the lint top, the tool treats the APB and pad signals as chip-level I/O. That is the origin of the unregistered-input warnings (W4–W9) and all six DFT warnings (D1–D6) below. In the SoC these are internal nets.

Front-End Messages

Reported during `analyze` and `elaborate`; not rule violations.

Message	Count	Meaning
`VERI-2418`	29	A `parameter` declared inside `gpio_reg_pkg` is treated as a `localparam`. This is what the SystemVerilog standard requires for package parameters; the message only reports it. Benign, generator style.
`VERI-1209`	1	`gpio_reg_top.sv` line 57: the 32-bit bus address is truncated to 11 bits (`assign reg_addr = reg_intf_req.addr`). **This is the root cause of B6**, reported here independently of the simulation. No action in the IP; the 2 KB SoC window makes the aliased addresses unreachable.

Results

Pass	Domain / category	Error	Warning	Info	Waived
1: LINT	FILEFORMAT	0	1	0	0
1: LINT	STRUCTURAL	0	8	0	0
2: all	DFT_FUNCTIONAL	0	6	0	0
2: all	LINT (same 9 as pass 1)	0	9	0	0
Total		**0**	**15**	**0**	**0**

Lint result

0 ERRORS / 15 WARNINGS / all 15 waived

No violation is reported in `gpio.sv`, `gpio_apb_wrap.sv` or `gpio_input_stage_no_clk_gates.sv`. Every warning is on the `gpio_lint_top` boundary or in a vendored file.

The LINT report is identical between the first single-pass run and pass 1 of the two-pass run, so the result is reproducible.

Note

Console summary count. During `check_superlint -extract` the console prints `Violation Count: Errors = 1 Warnings = 189 Info = 0`. That line is a raw count taken before the tool groups violations; every reported row names a whole bus, and a per-bit count of the 15 rows plus the 30 front-end messages comes to roughly 189. The single error in that line does not correspond to any violation in any report, including the all-domains report. The reports are the tool's authoritative output and record 0 errors; the summary-line error is recorded here as an unexplained discrepancy in the console count, not as a finding.

Waivers — LINT Domain

ID	Tag	Object	Justification
W1	FIL_MS_DUNM	Interface REG_BUS in reg_intf.sv	Naming style: interface name differs from file name. Vendored PULP register_interface code; renaming it would diverge from upstream for no functional gain.
W2	CMB_NR_TLIO	paddr_i[10:0] → prdata_o	By design. apb_to_reg is a feedthrough bridge and gpio_reg_top decodes the address into the read mux combinationally; this is how the slave answers with zero wait states. In the SoC the path runs from axi_lite_to_apb through the GPIO decode and back, and will appear in synthesis timing reports.
W3	CMB_NR_TLIO	paddr_i[10:0] → pslverr_o	Same as W2: the unmapped-address error is decoded combinationally from the address.
W4–W9	MOD_NO_IPRG	paddr_i, psel_i, penable_i, pwrite_i, pwdata_i, pstrb_i	Top-level inputs not registered. An APB slave samples its request on the clock edge; it does not register it on entry. These are internal SoC nets that only appear as top-level inputs because of the lint wrapper. Note that gpio_in_i is correctly *not* flagged: every pin passes through the 2-flop synchroniser.

Waivers — DFT Domain

All six are OTP_NO_RGTM: a top-level output is not driven directly by a register. DFT boundary rules are aimed at chip I/O; at block level these are internal SoC nets.

ID	Output	Justification
D1–D3	prdata_o, pready_o, pslverr_o	The APB response is combinational by design, for zero wait states (same root as W2, W3).
D4–D5	global_interrupt_o, pin_level_interrupts_o	Combinational from the edge detectors or the status registers, depending on CFG. The global line goes to the core in the same clock domain and is sampled synchronously there; the per-pin lines are unconnected at SoC level.
D6	gpio_tx_en_o	A mux of the GPIO_MODE and GPIO_OUT registers. **Waived with a condition**: on a mode change the mux can produce a short static hazard on the pad output enable, depending on how synthesis maps it. One APB write changes only one of the two registers, which keeps the exposure small. If the pad ring or IO specification requires a glitch-free output enable, register gpio_tx_en_o in gpio.sv instead, at the cost of one cycle from register write to pad.

Conclusion

The GPIO is lint-clean for the LINT sign-off rule set: 0 errors, and 9 warnings all waived with the reasons above. Enabling every rule domain adds 6 DFT warnings, also waived, and no errors. No lint finding requires an RTL change. The only conditional item is D6, which depends on the pad ring's requirements.

Verification Status

Verified

At block level, seed 1, against `gpio_apb_wrap`:

Feature	Evidence
Reset values of all registers and outputs	TC01, TC20
Register read/write, RO/WO semantics, unmapped-address errors	TC02, TC03, TC04
APB protocol: zero wait states, back-to-back, PSLVERR timing, no X	TC06, all four assertions
All four pin modes and SET/CLEAR/TOGGLE	TC07, TC08, TC09
Input synchronisation latency and `GPIO_EN` gating	TC10, TC11, TC12
All four interrupt types, timing, status, W1C and both `CFG` modes	TC13–TC17
Fixes B1–B5	TC18, TC19, TC22
Mixed random activity	TC21
Lint: 0 errors, 15 warnings, all waived	JasperGold Superlint, see Lint — JasperGold Superlint

Not Verified

Item	How to Close It
B6 at SoC level	SoC simulation: access `0x4000_3800`, expect SLVERR and no change to `GPIO_OUT`
Core response to a rejected sub-word store	SoC test with an `SB` to a GPIO register; confirm the core takes a store access fault and does not stall. Trace the SLVERR path in `hybrid_mem_interface.sv`
GPIO interrupt reaching the core	SoC test: edge on a pin, confirm `mip[16]`, trap with `mcause` 16, handler clears status
Access through the full AXI → APB path	Any SoC-level GPIO test; the block testbench drives APB directly
More than one seed	Re-run with several `+SEED` values; only seed 1 has been run
Coverage	No functional or code coverage has been collected
Clock-gated input stage	`gpio_input_stage.sv` is not compiled in this flow and has not been simulated
CDC, gate level	Not run. RTL simulation cannot model synchroniser metastability. (Lint has been run; see Lint — JasperGold Superlint.)
`gpio_axi_lite_wrap.sv`	Unused in this SoC; not verified

Open Items

- **Copy the SoC change to CentOS.** `riscforge_soc_subsystem.sv` is not compiled by the block filelist, so none of the runs above show whether the 2 KB window change has reached the CentOS tree.
- **Correct ARCH-INTEG.** Its GPIO subsection has the wrong register offsets, calls `GPIO_EN` an output enable, lists the level status registers as read-only, and gives the window as 4 KB.
- **Firmware rules** in the Programming Guide must be reflected in the GPIO driver: 32-bit stores only, clear status after enabling pins.

Related Finding: I2C Acknowledge Race

The same defect class as B4 exists, independently, in the I2C master:

Listing 7: `apb_i2c.sv`, line 183

```
irq_flag <= (s_done | i2c_al | irq_flag) & ~iack;
```

If a transfer completes or arbitration is lost on the same clock edge that software writes `IACK`, the `~iack` term clears the flag and the event is dropped. The B4 fix does not reach it; it is hand-written logic, not a `reggen` register. A `TODO` comment has been added above the line with a candidate fix, `irq_flag <= s_done | i2c_al | (irq_flag & ~iack)`. It has not been applied: whether it is correct depends on whether `s_done` is a single-cycle pulse, and it must be checked against the I2C testbench first.

Appendix: Superlint Script

`lint.tcl`, run from `rtl/gpio-master` with `jg -superlint lint.tcl`. It reads `gpio_lint.f` and elaborates `gpio_lint_top`; both sit in the same directory.

Listing 8: `lint.tcl`

```
# ============================================================================
# JasperGold Superlint - GPIO (gpio_apb_wrap, via gpio_lint_top)
#
# Run from rtl/gpio-master, the same directory the Xcelium testbench runs from:
#   jg -superlint lint.tcl
#
# Two passes, each a complete run, each with its own report:
#   1. LINT domain only - the sign-off report, same rule set as the I2C flow
#        reports/gpio_superlint_lint.txt
#   2. All rule domains - diagnostic; shows what pass 1 filters out, including
#      violations counted in the extraction summary but not in the LINT report
#        reports/gpio_superlint_all_domains.txt
# ============================================================================

proc run_superlint {domain_cmds report} {
  clear -all

  # config_rtlds is rejected (ERTL043) until an app is initialised
  check_superlint -init

  foreach cmd $domain_cmds {
    eval $cmd
  }

  analyze -sv -f gpio_lint.f
  elaborate -top gpio_lint_top

  clock clk_i
  reset ~rst_ni

  check_superlint -extract -top
  check_superlint -report -violation -detailed -file $report -force
}

file mkdir reports

# Pass 1: LINT domain only
run_superlint {
  {config_rtlds -rule -disable -domain {all}}
  {config_rtlds -rule -enable -domain {LINT}}
} reports/gpio_superlint_lint.txt

# Pass 2: every rule domain
run_superlint {
  {config_rtlds -rule -enable -domain {all}}
} reports/gpio_superlint_all_domains.txt
```